Mobile Application Development – CA1: Review 1
Idea & Proposal Review

Milestone: Problem definition, concept, and project scope

Section	Details
Group Number	Team 9
Team Members	GAYATHRI R – 25MCAR0093 HARSHINI M – 25MCAR0095
Title	MediLocker: Smart Medical Record Management with an AI-Based Health Information Assistant
App Name	MediLocker
Problem Statement	Medical documents such as prescriptions, laboratory reports, scan reports, discharge summaries, and medical bills are often kept in different places. This can make it difficult for people to find the correct record when they need it for a consultation or personal reference. MediLocker provides a mobile place to organise medical-record details and access them more conveniently. It also offers general health-information explanations through an AI assistant.
Abstract	MediLocker is an Android mobile application developed using Kotlin and XML in Android Studio. It provides a login screen, a dashboard, medical-record categories, record upload and listing screens, record details, document preview, alerts, and a profile screen. Record information and selected document URIs are stored locally using SharedPreferences. An AI medical-information chatbot connects to a Python Flask backend, which sends questions to the Gemini API and returns general explanations. The application aims to make personal record organisation easier. The AI feature is intended for general information only and does not replace professional medical advice.
Target Users	Individuals who want to organise and retrieve their personal medical documents, patients and caregivers who need convenient access to record details, and users who want simple explanations of common health terms. The current version is a prototype and should not be treated as a clinical record system.
Planned Core Features	1. Add and organise medical records by category. 2. View saved records and their details. 3. Select medical documents using the Android document picker and retain the selected URI. 4. Use the MediLocker AI chatbot for general health-information questions. 5. Access profile and alerts screens.
App Icon	

Remarks:

GAYATHRI R – 25MCAR0093
HARSHINI M – 25MCAR0095

Signature of Members

06/11/25

Signature of Faculty In charge
Mr. Aldrin Jose Antony

Mobile Application Development – CA1: Review 2

UI/UX Design Review

Milestone: Wireframes, mockups, and visual design language

Section	Details
Group Number	Team 9
Team Members	GAYATHRI R – 25MCAR0093 HARSHINI M – 25MCAR0095
Title	MediLocker: Smart Medical Record Management with an AI-Based Health Information Assistant
App Name	MediLocker
Screens Designed	Login; Dashboard/Home; Upload Medical Record; Records list; Record Details; Document Preview; Alerts; Profile; MediLocker AI chat.
Wireframe / Mockup Tool	The interface was implemented and previewed using Android Studio layout XML files and the Android emulator. No separate Figma/Adobe XD mockup link is currently documented. Emulator screenshots can be attached as visual evidence.
Colour Palette & Typography	A light, clean visual theme is used with a pale blue-grey background, blue primary buttons and highlights, dark navy headings, white cards, and small teal accents in the branding. Text uses readable Android sans-serif typography, with larger bold headings and smaller supporting text.
Design Consistency Notes	Screens use a consistent blue-and-white theme, familiar button styles, clear headings, and readable text. Related information is grouped into cards. Input fields have visible borders and controls have clear labels. The AI screen includes a note that responses are for general information and do not replace a doctor.
Layout & Visual Balance	Margins and padding separate content into readable sections. Cards and buttons are aligned to the screen content area. Scrollable containers are used where content can exceed the visible height, including the records list and AI conversation. Layout dimensions use Android dp and text uses sp to support different display densities and text sizes.
Add-ons as per Remarks	The screens were refined during implementation to improve scrolling, spacing, navigation, and the visibility of interactive controls. The AI chat uses separate user and assistant message bubbles.

Evaluation – UI Design & Visual Appeal

Remarks:

GAYATHRI R – 25MCAR0093

HARSHINI M – 25MCAR0095

Signature of Members

06/11/25

Signature of Faculty In charge

Mr. Aldrin Jose Antony

Mobile Application Development – CA1: Review 3

Development & Responsiveness Review

Milestone: Core build progress and responsive layout implementation

Section	Details
Group Number	Team 9
Team Members	GAYATHRI R – 25MCAR0093 HARSHINI M – 25MCAR0095
Title	MediLocker: Smart Medical Record Management with an AI-Based Health Information Assistant
App Name	MediLocker
Features Implemented	Login screen with demo validation; dashboard with greeting and summary; medical-record categories; upload flow using the Android document picker; local saving and display of record details; records list and record-detail navigation; Alerts and Profile screens; MediLocker AI chat connected to a Flask/Gemini backend. These features have been run and checked in the development emulator.
Tech Stack Used	Android Studio; Kotlin; XML layouts; Android SDK; Material/Android UI components; SharedPreferences with JSON for local record metadata; Android document picker for file selection; Python 3.11; Flask; google-genai SDK; python-dotenv; Gemini API; HTTP/JSON communication between the Android app and the local Flask backend.
Devices / Screen Sizes Tested	Android Studio emulator configured as Pixel 5. Testing has primarily been performed in this emulator; testing on a range of physical devices and screen sizes is still required.
Responsive Design Approach	Layouts use XML with ConstraintLayout and LinearLayout, dp-based spacing, sp-based text sizes, and ScrollView containers for longer content. The chat screen adds message bubbles dynamically and scrolls to the latest message. Additional testing on smaller and larger displays is recommended.
Known Issues / Bugs	The AI service can occasionally time out or return a temporary provider-unavailable error. The Flask backend must be running on the development computer for emulator chat requests to work. The current version stores record metadata and document URIs locally; it does not provide cloud synchronisation, server-side medical-record storage, or a verified encryption/security framework. Production deployment and broader device testing remain pending.
Add-ons as per Remarks	The AI model name was checked against models available to the configured API key, the local chat request was tested successfully, duplicate welcome text was removed, and the chat UI was refined. Intermittent external AI-service failures remain a dependency rather than a confirmed UI defect.

Evaluation – Responsiveness & Interactivity:

Remarks:

GAYATHRI R – 25MCAR0093

HARSHINI M – 25MCAR0095

Signature of Members

06/11/25

Signature of Faculty In charge

Mr. Aldrin Jose Antony

Mobile Application Development – CA1: Review 4
UX, Navigation & Usability Testing

Milestone: User flow, interactivity, and usability feedback

Section	Details
Group Number	Team 9
Team Members	GAYATHRI R – 25MCAR0093 HARSHINI M – 25MCAR0095
Title	MediLocker: Smart Medical Record Management with an AI-Based Health Information Assistant
App Name	MediLocker
Navigation Flow Summary	After login, users reach the Dashboard. The bottom navigation provides access to Home, Records, Alerts, and Profile. The Dashboard also provides shortcuts to record categories and the Add Medical Record flow. Selecting a saved record opens its details or preview where supported. The Ask MediLocker AI button opens the chatbot, where users enter a question and receive a response from the Flask/Gemini service.
Interactive Components Used	Email and password input fields; login and navigation buttons; dashboard category cards; search field; Add Medical Record button; document picker; record cards; back navigation; alerts and profile controls; AI message input; Send button; scrollable chat history; selectable message text; toast messages for input or connection feedback.
Usability Testing Method	Informal functional testing was carried out by running the app in the Android Studio Pixel 5 emulator. Main flows were checked, including opening screens, navigating back, adding a record, viewing the records list, opening Profile and Alerts, and sending questions to the AI backend. A formal study with external users has not yet been documented.
Usability Feedback Received	Development checks identified a need to fix the Records screen scrolling and back navigation, make the AI entry point easier to find, remove duplicate welcome text, and handle long AI replies and service errors. These items were addressed in the current build where applicable. Further feedback should be collected from representative users before final release.
Navigation Improvements Planned	Test navigation on multiple screen sizes; improve keyboard and long-response behaviour in the chat; provide clearer retry feedback when the AI service is unavailable; and review the upload and record preview flow with users. Add production-grade authentication and secure storage before using real medical documents.
Add-ons as per Remarks	The Records back action and scrolling were corrected, and the AI chat was linked from the app. The chat now displays user and assistant messages in separate bubbles and shows connection errors when a request fails.

Evaluation – User Experience & Navigation

Remarks:

GAYATHRI R – 25MCAR0093

HARSHINI M – 25MCAR0095

Signature of Members

06/11/25

Signature of Faculty In charge

Mr. Aldrin Jose Antony

Mobile Application Development – CA1: Review 5
Final Presentation & Deployment Review

Milestone: Completed app, deployment, and final demo

Section	Details
Group Number	Team 9
Team Members	GAYATHRI R – 25MCAR0093 HARSHINI M – 25MCAR0095
Title	MediLocker: Smart Medical Record Management with an AI-Based Health Information Assistant
App Name	MediLocker
Deployment Details	Current status: development build running in the Android Studio emulator. The AI backend runs locally using Python Flask and requires the development computer and Gemini API connection to be available. The app has not been confirmed as published to the Google Play Store or deployed to a production server. APK distribution and production HTTPS hosting are pending.
Final Feature Summary	The current prototype includes login, a dashboard, medical-record categories, adding and listing records, locally saved record metadata and document URIs, record details/preview navigation, Alerts, Profile, and a MediLocker AI chatbot that sends general health questions to a Flask backend and Gemini API. The AI feature includes a general-information disclaimer. Features requiring cloud storage, verified encryption, and production deployment are not included in the current build.
Demo Notes	Demonstrate the login flow; dashboard and category shortcuts; add a sample non-sensitive record; open the Records list and record details; visit Alerts and Profile; open MediLocker AI and ask a simple general health question; show the response and explain that the backend must be running. Do not use real patient information during the demonstration.
Project Report Submitted	In preparation — this report template is being completed. Update this field after the final report has been submitted.
App Icon	MediLocker logo/app icon is included in the Android project assets.
Add-ons as per Remarks	Before final submission, perform broader device testing, verify the final APK, document test results, and decide whether the backend will remain a local demonstration or be deployed securely using HTTPS.

Final Evaluation – CA1 Rubric Total 15 Marks

Remarks:

GAYATHRI R – 25MCAR0093

HARSHINI M – 25MCAR0095

06/11/26

Signature of Members

Signature of Faculty In charge
Mr. Aldrin Jose Antony